

WEEK 6

Q1: Explain the roles of the **Driver**, **Cluster Manager**, and **Executor** in a Spark application.

Ans-

Driver: The main program that creates the SparkSession, builds the execution plan (DAG), schedules tasks, and collects the results.

Cluster Manager: Manages cluster resources and allocates them to Spark applications. Examples include Standalone, YARN, Kubernetes, and Mesos.

Executor: Worker processes that execute tasks assigned by the Driver, store data in memory, and return results to the Driver.

Q2: How does Spark's **Lazy Evaluation** strategy improve performance when chain-processing large datasets?

Ans-

- i. Spark does not execute transformations immediately. Instead, it records all transformations in a **Directed Acyclic Graph (DAG)** and waits until an action (such as `show()` or `count()`) is called.
- ii. This allows Spark to optimize the execution plan, reduce unnecessary computations, minimize data movement, and improve overall performance.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and `inferSchema` is enabled.

Ans-

```
df = spark.read.csv(  
    "data/source.csv",  
    header=True,  
    inferSchema=True  
)
```

Q4: What is the difference between **CSV** and **Parquet** in terms of storage (row-based vs. columnar) and why does it matter for performance?

Ans-

Aspect	CSV	Parquet
Storage Format	Row-based	Columnar
File Size	Larger	Smaller due to compression
Compression	Limited/No built-in compression	Built-in compression supported
Read Performance	Slower for analytics	Faster for analytical queries
Write Performance	Generally faster	Slightly slower due to encoding/compression
Schema Support	No schema metadata	Stores schema metadata
Predicate Pushdown	Not supported	Supported

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

Ans- from pyspark.sql.functions import col

```
selected_df = df.filter(col("product_category") == "Electronics") \
    .select("user_id", "price")
```

```
selected_df.show()
```

```
... +-----+-----+
    |user_id|price|
    +-----+-----+
    |1001| NULL|
    |1005| 120|
    |1009| 140|
    |1013| 160|
    |1017| 180|
    |1021| 200|
    |1025| 220|
    |1029| 240|
    |1033| 260|
    |1037| 280|
    |1041| 300|
    |1045| 320|
    |1049| 340|
    |1053| 360|
    |1057| 380|
    |1061| 400|
    |1065| 420|
    |1069| NULL|
    |1073| 460|
    |1077| 480|
    +-----+-----+
only showing top 20 rows
```

Q6: Write the code to "revise" a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.

Ans- from pyspark.sql.functions import col

```
df = df.withColumnRenamed("old_name", "new_name")
```

```
df = df.withColumnn(
```

```
    "price",
```

```
    col("price").cast("double")
```

```
)
```

```
df.show()
```

...	user_id	transaction_date	region	product_category	sale_amount	city	age	subscription	price	store_id	email	username
	1001	01-01-2025	West	Electronics	100	Delhi	18	Premium	NULL	S101	NULL	NULL
	1002	02-01-2025	West	Furniture	105	Delhi	19	Basic	105.0	S101	user1002@gmail.com	user1002
	1003	03-01-2025	West	Clothing	110	Delhi	20	Premium	110.0	S101	user1003@gmail.com	user1003
	1004	04-01-2025	West	Grocery	115	Delhi	21	Basic	115.0	S101	user1004@gmail.com	user1004
	1005	05-01-2025	West	Electronics	120	Delhi	22	Premium	120.0	S101	user1005@gmail.com	user1005
	1006	06-01-2025	West	Furniture	125	Delhi	23	Basic	125.0	S101	user1006@gmail.com	user1006
	1007	07-01-2025	West	Clothing	130	Delhi	24	Premium	130.0	S101	user1007@gmail.com	user1007
	1008	08-01-2025	West	Grocery	135	Delhi	25	Basic	135.0	S101	user1008@gmail.com	user1008
	1009	09-01-2025	West	Electronics	140	Delhi	26	Premium	140.0	S101	user1009@gmail.com	user1009
	1010	10-01-2025	West	Furniture	145	Delhi	27	Basic	145.0	S101	user1010@gmail.com	user1010
	1011	11-01-2025	West	Clothing	150	Delhi	28	Premium	150.0	S101	user1011@gmail.com	user1011
	1012	12-01-2025	West	Grocery	155	Delhi	29	Basic	155.0	S101	user1012@gmail.com	user1012
	1013	13-01-2025	West	Electronics	160	Delhi	30	Premium	160.0	S101	user1013@gmail.com	user1013
	1014	14-01-2025	West	Furniture	165	Delhi	31	Basic	165.0	S101	user1014@gmail.com	user1014
	1015	15-01-2025	West	Clothing	170	Delhi	32	Premium	170.0	S101	user1015@gmail.com	user1015
	1016	16-01-2025	West	Grocery	175	Delhi	33	Basic	175.0	S101	user1016@gmail.com	user1016
	1017	17-01-2025	West	Electronics	180	Delhi	34	Premium	180.0	S101	user1017@gmail.com	user1017
	1018	18-01-2025	West	Furniture	185	Delhi	35	Basic	NULL	S101	user1018@gmail.com	user1018
	1019	19-01-2025	West	Clothing	190	Delhi	18	Premium	190.0	S101	user1019@gmail.com	user1019
	1020	20-01-2025	West	Grocery	195	Delhi	19	Basic	195.0	S101	NULL	user1020

only showing top 20 rows

Q7: How does Spark use the **Lineage Graph (DAG)** to provide fault tolerance if a worker node fails?

Ans-

- Spark maintains a **Lineage Graph (DAG)** that records all transformations performed on the data.
- If an executor or worker node fails, Spark uses this lineage information to recompute only the lost partitions instead of reprocessing the entire dataset.
- This provides efficient fault tolerance without requiring full data replication.

Q8: Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

Ans- from pyspark.sql.functions import col

```
df_orders.filter(
```

```
    (col("status") == "Completed") &
```

```
    (col("amount") > 1000)
```

).show()

Q9: Explain the concept of **Predicate Pushdown** in Parquet and how it affects the amount of data loaded into memory.

Ans-

- a. **Predicate Pushdown** is a Spark optimization technique supported by the **Parquet** file format.
- b. It pushes filter conditions (predicates) down to the storage layer before reading the data.
- c. Instead of scanning the entire dataset, Spark reads only the rows or row groups that satisfy the filter condition.
- d. This reduces the amount of unnecessary data read from disk.
- e. Since less data is loaded into memory, memory usage is significantly reduced.
- f. It decreases disk I/O operations, improving overall query performance.
- g. Predicate Pushdown is especially beneficial when working with large datasets containing millions of records.
- h. It helps Spark execute filtering operations faster by skipping irrelevant data.
- i. This optimization is available in columnar file formats like **Parquet** and **ORC**, but not in plain CSV files.

Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

Ans- `from pyspark.sql.functions import col`

```
df = df.withColumn(  
    "final_price",  
    col("base_price") * 1.18  
)
```

Q11: What is the difference between **Transformations** and **Actions**? Provide two examples of each.

Ans –

Aspect	Transformations	Actions
Definition	Operations that create a new DataFrame/RDD	Operations that trigger execution and return results
Execution	Lazy (not executed immediately)	Immediate execution

Aspect	Transformations	Actions
Purpose	Modify or transform data	Retrieve results or save data
Return Type	New DataFrame/RDD	Value or output
DAG Creation	Adds operations to the DAG	Executes the DAG
Performance	Optimized before execution	Executes optimized plan
Examples	filter(), select(), withColumn(), groupBy()	show(), count(), collect(), first()

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

Ans- from pyspark.sql.functions import col

```
spark.read.parquet("path/to/input") \
    .filter(col("user_id").isNotNull()) \
    .write.mode("overwrite") \
    .option("header", True) \
    .csv("path/to/output")
```

Q13: In Spark Architecture, what is the difference between **Client Mode** and **Cluster Mode**?

Ans-

Aspect	Client Mode	Cluster Mode
Driver Location	Runs on the client machine	Runs inside the cluster
Resource Management	Executors run in the cluster, Driver remains local	Both Driver and Executors run in the cluster
Network Dependency	Requires continuous connection between client and cluster	Client can disconnect after job submission
Fault Tolerance	Driver failure stops the application	Driver managed by the cluster, providing better reliability

Aspect	Client Mode	Cluster Mode
Performance	Suitable for development and testing	Suitable for production workloads
Use Case	Interactive analysis, debugging	Large-scale production jobs

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

Ans- from pyspark.sql.functions import col

```
df.filter(
    (col("region") == "North") |
    (col("priority") == "High")
).show()
```

Q15: When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

Ans-

Aspect	.show()	.collect()
Purpose	Displays a few rows	Retrieves all rows
Data Returned	Default first 20 rows (or specified number)	Entire dataset
Memory Usage	Low	High
Suitable for Large Datasets	Yes	No
Risk	Safe	Can cause OutOfMemory errors
Return Type	Prints formatted output	Returns a Python list of Row objects
Typical Use	Previewing data	Further processing in the driver program